МЕТОДИЧЕСКИЕ УКАЗАНИЯ

к практическим занятиям и самостоятельной работе

по дисциплине

Б1.О.25.01 Основы алгоритмизации и программирование

Направление подготовки
13.03.02 Электроэнергетика и электротехника

Профиль подготовки
Электроснабжение

Квалификация (степень) выпускника
бакалавр

Уфа 2024

УДК 621.31:631(075.8)
ББК 40.76

Рекомендовано к изданию методической комиссией энергетического факультета (протокол № 7 от 21.03.2024 г.)

Составитель: д.т.н., профессор Галиуллин Р.Р.

Рецензент: к.т.н., доцент Кафиев И.Р.

Ответственный за выпуск: зав. кафедрой ЭАТП к.т.н., доцент Ахметшин А.Т.

г. Уфа, БашГАУ, Кафедра электроснабжения и автоматизации технологических процессов

ОГЛАВЛЕНИЕ

	Введение	4
1	Алгоритмы и способы их описания	5
2	Базовые структуры алгоритмов	8
3	Знакомство с инструментальной средой программирования	17
4	Разработка программ линейной структуры	21
5	Разработка программ разветвляющейся структуры	24
6	Разработка программ циклической структуры	27
7	Обработка одномерных числовых массивов. Сортировка одномерных массивов	30
8	Разработка программ с использованием двумерных массивов. сортировка двумерных массивов	33
9	Обработка строк. Функции и процедуры для работы со строками	36
10	Процедуры и функции	40
	Библиографический список	43

ВВЕДЕНИЕ

Дисциплина «Основы алгоритмизации и программирование» ведется в рамках модуля «Информационные технологии и программирование» составляет основу для подготовки специалистов по направлению «Электроэнергетика и электротехника».

Целью изучения дисциплины является формирование у будущего инженера системы знаний и практических навыков, необходимых для решения основных задач, связанных с применением алгоритмов и компьютерных программ в электроснабжении.

Задача дисциплины «Основы алгоритмизации и программирование» вытекают из требований ФГОС и квалифицированной характеристики выпускника и сводятся к изучению основных задач электроснабжения с применением алгоритмов и компьютерных программ.

Изучение дисциплины «Основы алгоритмизации и программирование» предполагает предварительное освоение следующих дисциплин в рамках общеобразовательного курса: основы информатики и математика.

Методические указания к практическим занятиям по основам алгоритмизации и программированию составлены в соответствии с типовой и рабочей программой дисциплины.

Материалы по всем темам составлены на основе действующих библиографических источников и нормативных документов в области систем электроснабжения.

В результате решения практических задач по дисциплине «Основы алгоритмизации и программирование» студенты должны приобрести навыки и умения по разработке алгоритмов и компьютерных программ, пригодные для практического применения в системах электроснабжения.

1 АЛГОРИТМЫ И СПОСОБЫ ИХ ОПИСАНИЯ

Цель занятия – формирование знаний, об алгоритмах и способах их описания.

Общие сведения

Алгоритм – понятное и точное предписание исполнителю совершить последовательность действий, направленных на достижение цели.

Любой алгоритм должен обладать следующими свойствами:

– определенностью - за конечное число шагов либо должен быть получен результат, либо доказано его отсутствие;

– результативностью - обязательным получением некоторого результата (числа, таблицы, текста, звука, изображения и т. д.) или сигнала о том, что данный алгоритм неприменим для решения поставленной задачи;

– массовостью - возможностью получения результата при различных исходных данных для некоторого класса сходных задач;

– формальностью - отвлечение от содержания поставленной задачи и строгое выполнение некоторого правила, инструкции;

– дискретностью - возможностью разбиения алгоритма на отдельные элементарные действия.

Существуют следующие формы представления алгоритма:

– словесно-формульная на неформальном языке;

– на языках программирования;

– графическая.

Словесная форма представления алгоритма имеет ряд недостатков. Для достаточно сложных алгоритмов описание становится слишком громоздким и не наглядным. Эта форма представления обычно используется лишь на начальных стадиях разработки алгоритма.

Пример. Составим формульно-словесный алгоритм вычисления выражения: $z=2\cdot x-(y+6)$:

– вводим значения x и y;

– находим сумму (y+6);

– находим произведение ($2\cdot x$);

– вычисляем z как разность уже полученных выше значений: $z=2\cdot x-(y+6)$;

– выводим z как результат вычисления выражения.

Это более компактный и лаконичный метод, он нагляднее, но всё же строго формальным не является.

Алгоритм, записанный на языке программирования, называется программой. Программа представляет собой алгоритм, который записан как последовательность команд. Речь идёт о командах, понятных компьютеру, для чего используются различные языки программирования, представляющие собой системы кодирования предписаний с правилами их применения. Языки программирования характеризуются строго определённым синтаксисом, то есть свободное толкование конструкций не допускается.

Графическая запись алгоритма – описание структуры алгоритма с помощью геометрических фигур с линиями связями, показывающими порядок выполнения отдельных инструкций. Описание алгоритмов с помощью схем – один из наиболее наглядных и компактных способов. Этот способ имеет ряд преимуществ перед остальными: наглядное отображение базовых конструкций алгоритма; концентрация внимания на структуре алгоритма; использование принципа блочности при коллективном решении сложной задачи; преобразование алгоритма методом укрупнения (сведения к единому блоку) или детализации (разбиения на ряд блоков); быстрая проверка разработанного алгоритма.

Условные графические изображения, используемые при построении схем, называются символами. Система символов и правила построения алгоритмов определены соответствующими стандартами: блок-схема выстраивается в одном направлении: либо сверху вниз, либо слева направо, в порядке выполнения действий. Для наглядности операции разного вида изображаются в схеме различными геометрическими фигурами таблица 1.

Таблица 1 – Средства графического изображения алгоритмов

Наименование	Символ	Выполняемые действия
Терминатор (пуск-остановка)		Обозначает начало и конец алгоритма
Данные (ввод-вывод)		Ввод и вывод данных
Процесс		Вычислительные действия
Решение		Наличие условия в алгоритме
Граница цикла		Наличие цикла в алгоритме
Предопределенный процесс		Наличие подпрограммы
Соединитель		Разрыв алгоритма (соединительные блоки)
Комментарий		Внесение комментариев

Терминатор (пуск-остановка). Элемент отображает вход из внешней среды или выход из нее (наиболее частое применение − начало и конец алгоритма). Внутри фигуры записывается соответствующее действие − начало/конец.

Данные (ввод-вывод). Преобразование данных в форму, пригодную для обработки (ввод) или отображения результатов обработки (вывод). Данный символ не определяет типа носителя данных.

Процесс. Выполнение одной или нескольких операций, обработка данных любого вида (изменение значения данных, формы представления, расположения). Внутри фигуры записывают непосредственно сами операции. В этом блоке знак «=» означает не математическое равенство, а операцию присваивания.

Решение. Отображает решение или функцию переключательного типа с одним входом и двумя или более альтернативными выходами, из которых только один может быть выбран после вычисления условий, определенных внутри этого элемента. Вход в элемент обозначается линией, входящей обычно в верхнюю вершину элемента. Если выходов два или три, то обычно каждый выход обозначается линией, выходящей из оставшихся вершин. Если выходов больше трех, то их следует показывать одной линией.

Граница цикла. Условия цикла и приращения записываются внутри символа цикла − в зависимости от типа организации цикла. Для изображения итерационного цикла на блок-схеме вместо данного символа используют символ решения, указывая в нем условие, а одну из линий выхода замыкают выше в блок-схеме.

Предопределенный процесс. Символ отображает выполнение процесса, состоящего из одной или нескольких операций, который определен в другом месте программы. Внутри символа записывается название процесса и передаваемые в него данные.

Соединитель. Используется для обрыва линии и продолжения ее в другом месте. Пример: разделение блок-схемы, не помещающейся на листе. Соответствующие соединительные символы должны иметь одно обозначение.

Комментарий. Позволяет включать в схемы алгоритмов пояснения к функциональным блокам. Частое использование комментариев нежелательно, т. к. это усложняет схему, делает ее менее наглядной.

Вопросы для самоконтроля:

1. Дайте определение алгоритма.
2. Назовите основные преимущества графического описания алгоритмов.
3. Перечислите основные свойства алгоритмов и раскройте их сущность.

2 БАЗОВЫЕ СТРУКТУРЫ АЛГОРИТМОВ

Цель занятия – закрепление знаний, умений и навыков по основным алгоритмическим структурам: линейный алгоритм, алгоритм “ветвления” и циклический алгоритм, а также отработка навыков работы с графическим способом описания алгоритмов.

Общие сведения

Линейный алгоритм.

Любой алгоритм составляется из ряда базовых структур. Простейшей базовой структурой является следование - структура с линейными характеристиками. Из этого можно сформулировать определение.

Линейный алгоритм - это алгоритм, образуемый командами, которые выполняются однократно и именно в той последовательности, в которой записаны. Линейная структура, по сути, проста. Записать её можно как в текстовой, так и в графической форме.

Представим, что у нас стоит задача пропылесосить ковёр в комнате. В текстовой форме алгоритм будет следующим:

- принести пылесос к месту уборки;
- включить;
- пропылесосить;
- выключить;
- унести пылесос.

Алгоритм «Определение периметра треугольника» в графической форме с использованием блок-схем представлен на рисунке 1.

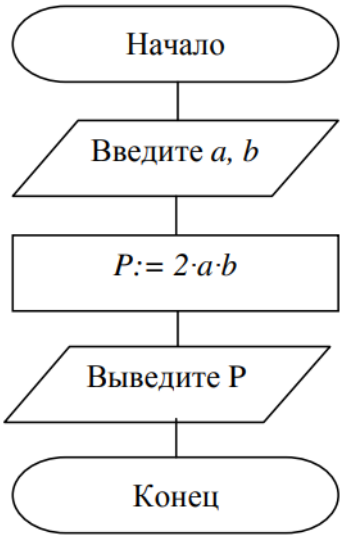


Рисунок 1 – Блок схема алгоритма «Определение периметра треугольника»

Пример 1. Создайте линейный алгоритм для вычисления площади прямоугольника по заданной длине и ширине.

Решение.

1) Ввести a и b.

2) Вычислить S = a*b.

3) Вывести полученный результат на экран.

4) Закончить выполнение алгоритма.

Пример 2. Построить блок-схему алгоритма, который для заданных a и b вычисляет:

$$\begin{cases} z = x^3 + y^3 + 10, \\ y = \sin(b) - \cos(a + x), \\ x = \sqrt{|a+1|}. \end{cases}$$

Решение дано на рисунке 2.

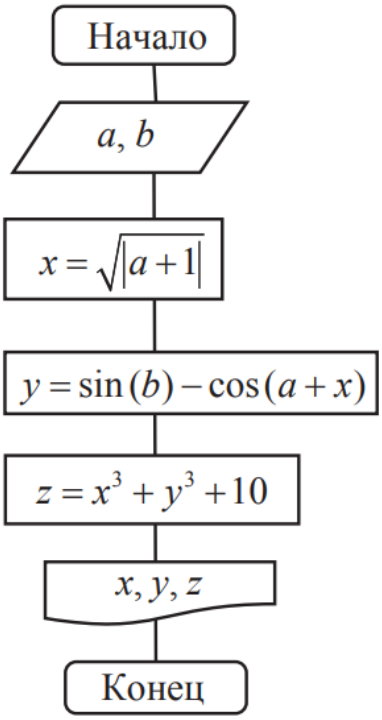


Рисунок 2 – Блок схема алгоритма к задаче 2

Задания для самостоятельной работы

1. Составить блок схему алгоритма вычисления периметра и площади квадрата со стороной a.

2. Даны два ненулевых числа. Найти их сумму, разность, произведение и частное.

3. Дана длина окружности. Найти площадь круга, ограниченного этой окружностью. В качестве значения Pi использовать 3.14.

4. Дана сторона равностороннего треугольника. Найти площадь этого треугольника и радиусы вписанной и описанной окружностей.

5. Найти корни квадратного уравнения Ax2 + Bx + C = 0, заданного своими коэффициентами A, B, C (коэффициент A не равен 0), если известно, что дискриминант уравнения неотрицателен.

6. Найти площадь кольца, внутренний радиус которого равен R1, а внешний радиус равен R2 (R1< R2).

7. Построить блок-схему алгоритма, который для заданных x, y, z позволяет вычислить a, b, если:

1) $a = \dfrac{\sqrt{|x-1|} - \sqrt[3]{|y|}}{1 + \dfrac{x^2}{2} + \dfrac{y^2}{4}}$, $\quad b = x\left[\operatorname{arctg}(z) + e^{-(x+3)}\right]$;

2) $a = \dfrac{3 + e^{y-1}}{1 + x^2\left|y - \operatorname{tg}(z)\right|}$, $\quad b = 1 + |y - x| + \dfrac{(y-x)^2}{2} + \dfrac{|y-x|^3}{3}$;

3) $a = \dfrac{2\cos\left(x - \dfrac{\pi}{6}\right)}{\dfrac{1}{2} + \sin^2(y)}$, $\quad b = 1 + \dfrac{z^2}{3 + \dfrac{z^2}{5}}$.

Алгоритм ветвления

В повседневной жизни таких ситуаций, в которых заранее известен алгоритм действий и результат, очень мало. Практически постоянно нам приходится принимать решения от которых будут зависеть дальнейшие действия.

Существуют полная и неполная формы ветвления.

В полной форме если условие выполняется, то алгоритм переходит к выполнению первой серии команд, а если не ко второй.

В неполной форме алгоритм выполняет серию команд только если условие истинно. В противном случае ничего не происходит.

Алгоритм, основанный на конструкции «ветвление» называется разветвляющимся алгоритмом.

Задания для самостоятельной работы

1. Ввести целое число N. Выяснить, кратно ли оно трём.

2. По заданным x, y составить программу вычисления значения z:

1) $z = \dfrac{\min(x, y) + 0{,}5}{1 + \max^2(x, y)}$;

2) $z = \begin{cases} \max(x, y) & \text{при } x < 0, \\ \min(x, y) & \text{при } x \geq 0; \end{cases}$

3) $z = \begin{cases} \min(x, y) & \text{при } y \geq 0, \\ \max(x^2, y^2) & \text{при } y < 0. \end{cases}$

3. Ввести два целых числа. Выяснить, являются ли они оба четными или нечетными, либо одно четное, а другое нечетное.

4. Даны действительные числа a, b, c. Проверить, выполняются ли неравенства a > b > c.

5. Даны действительные числа a, b, c. Удвоить эти числа, если
a < b < c.

6. Ввести два целых числа. Выяснить, являются ли они оба четными или нечетными, либо одно четное, а другое нечетное.

7. Даны два действительных числа. Заменить первое число нулем, если оно меньше второго или равно ему, и оставить числа без изменения в противном случае.

8. Даны три действительных числа. Выбрать из них те, которые принадлежат интервалу (1,3).

9. Даны действительные числа *x, y.* Определить, принадлежит ли точка с координатами (x, *y)* заштрихованной части плоскости в вариантах *а* - *в.*

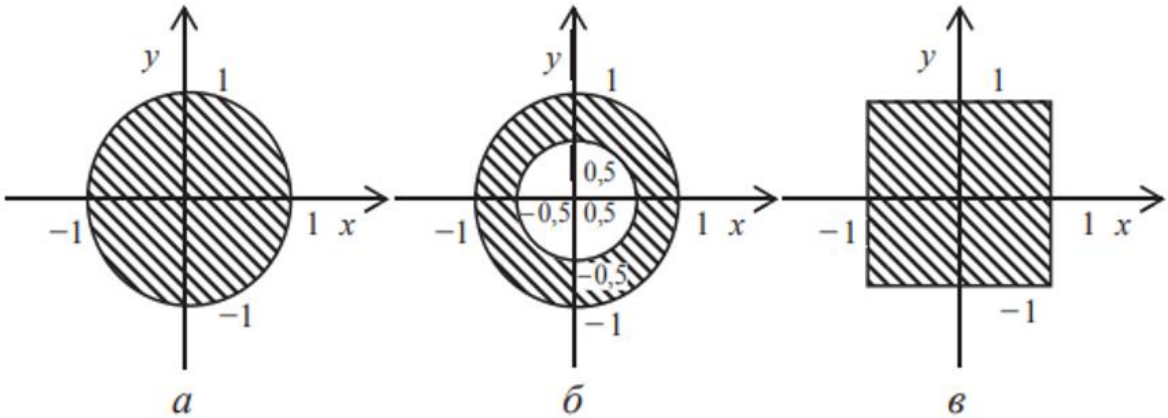


Рисунок 3 – Фигуры к заданию 9

Пояснения к заданию 18. Если расстояние меньше или равно радиусу окружности, то точка принадлежит окружности. Для вычисления расстояния между двумя точками можно использовать формулу:

d = sqrt((x2 - x1)^2 + (y2 - y1)^2),

где (x1, y1) - координаты центра окружности, (x2, y2) -координаты точки.

Если полученное расстояние меньше или равно радиусу окружности, то точка принадлежит окружности. В противном случае - нет.

10. Заданы целые числа x, y, *m, n.* Если разность (x-y) меньше остатка от деления *m* на *n*, увеличить x на 1.

11. Задано целое трехзначное число *К.* Определить, организуют ли цифры этого числа упорядоченную последовательность, и выдать соответствующее сообщение.

12. По четырехзначному номеру трамвайного билета определить, является ли он счастливым (билет считается счастливым, если сумма первых двух цифр номера совпадает с суммой двух его последних цифр).

13. Заданы целые числа: *к, I, n, m.* Проверить, является ли *к* делителем всех чисел.

14. Заданы размеры *A, B* прямоугольного отверстия и размеры x, *y, z* кирпича. Определить, проходит ли кирпич через отверстие.

15. Заданы числа *a, b, c.* Напечатать эти числа в порядке убывания их абсолютных величин.

16. Написать программу для вычисления значения функции *f* для заданного значения x:

$$1)\ f(x)=\begin{cases}4 & \text{при } x<0,\\ x^2+3x+4 & \text{при } 0\le x<1,\\ x+7 & \text{при } x>1;\end{cases}$$

$$2)\ f(x)=\begin{cases}\sin^2\left(x+\dfrac{\pi}{2}\right) & \text{при } x<0,\\ e^x\sin\left(x+\dfrac{\pi}{2}\right) & \text{при } a\le x\le b,\\ \sqrt{\left|\sin\left(x+\dfrac{\pi}{3}\right)\right|+2{,}1\cdot10^{-3}} & \text{при } x>b.\end{cases}$$

17. Заданы числа *к, I, m* (целые) и x. Вычислить величины y(x) и x(y):

$$y(x)=\begin{cases}\dfrac{\sin|x|}{\sqrt{x^2+1}}, & \text{если } m=\min(k,l),\\ |x|\ln(1+x), & \text{если } m=\max(k,l),\\ x^3+x+10^{-2}, & \text{если } m\in[\min(k,l),\max(k,l)],\\ -1 & \text{в остальных случаях;}\end{cases}$$

$$z(y)=y^4-y^2+5.$$

18. Выяснить, попадает ли точка с координатами (x_0,y_0) в кольцо, образованное окружностями $x^2+y^2=r^2$ и $x^2+y^2=R^2$ $(r<R)$.

19. Определить, в какой координатной четверти находится заданная точка с координатами (x, y).

20. Заданы числа x, *y, R.* Определить расстояние от точки с координатами (x, *y)* до контура полукруга радиуса *R* с центром в начале координат, расположенного в нижней полуплоскости.

21. Заданы координаты (x_0,y_0) точки на плоскости. Проверить, лежит ли точка в верхней полуплоскости, и выдать соответствующее сообщение.

22. Ввести три вещественных числа: *a, b, c*- длины трех отрезков. Если отрезки могут быть сторонами треугольника, найти его периметр и площадь.

23. Определить, является ли заданный год W високосным. Год високосный, если N не кратно 100 и число, изображаемое его двумя последними цифрами, кратно четырем. Если N кратно 100, то год високосный лишь при N, кратном 400.

24. Заданы координаты вершин прямоугольника со сторонами, параллельными осям координат. Определить площадь части прямоугольника, расположенной в первой координатной четверти.

Циклический алгоритм

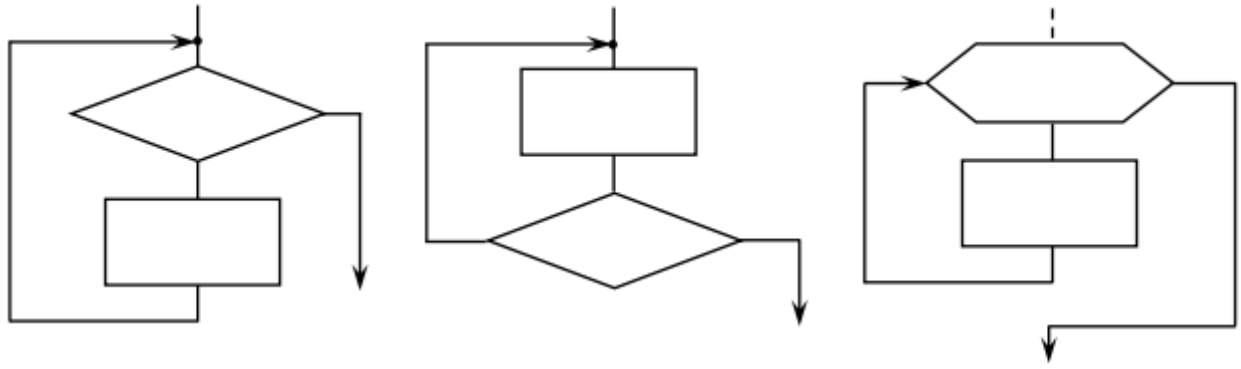
Алгоритм, содержащий конструкцию «повторение» называется циклическим алгоритмом.

Существует несколько разновидностей циклических алгоритмов (рисунок 4):

Первый - цикл с заданным условием продолжения работы (цикл с предусловием или цикл-пока).

Второй - цикл с заданным условием окончания работы (цикл с постусловием или цикл-до).

И третий - цикл с заданным числом повторений (цикл с параметром).



с предусловием с постусловием с постусловием

Рисунок 4 – Схемы представления циклических алгоритмов

Особенностью первой схемы (цикл с предусловием) является то, что проверка условия проводится до тела цикла. В том случае, если условие выхода из цикла выполняется, то тело цикла не выполняется ни разу.

При решении задачи по циклическому алгоритму с постусловием в любом случае цикл будет выполнен хотя бы один раз, так как первая проверка условия выхода из цикла осуществляется после того, как тело цикла выполнено.

В условии многих задач четко оговаривается количество повторений операций тела цикла или, например, задано выполнить действия для всех значений переменной от первого до последнего с заданным шагом изменении.

Параметр цикла – это переменная, через которую организуется (описывается) цикл. Параметр цикла определяет количество повторений (итераций), так как у него обязательно известно - первое значение, при котором тело цикла будет выполнено первый раз, последнее значение, при котором тело цикла будет выполнено последний раз и величина изменения параметра (шаг) на которую надо увеличить параметр после каждой итерации.

Шаг – числовой показатель изменения параметра после каждой итерации.

На практике при решении конкретной задачи параметром цикла чаще всего назначается переменная-счетчик, которая четко задает количество итераций.

Пример 1. С клавиатуры вводятся N натуральных чисел. Рассчитать, сумму чётных и произведение нечётных чисел.

Решение – блок схема представлена на рисунке 5.

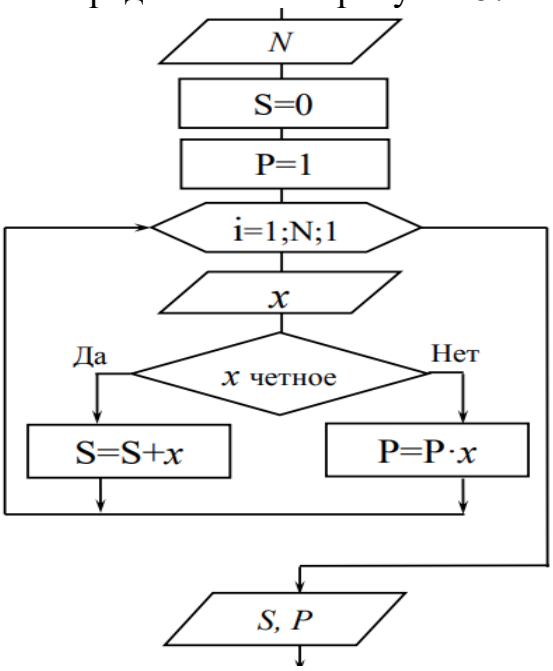


Рисунок 5 – Блок схема к примеру 1

Пример 2. Представить блок-схему алгоритма решения следующей задачи. Вывести все точки графика y=x2 на отрезке для X от -5 до 5 с шагом 1.

Решение– блок схема представлена на рисунке 6.

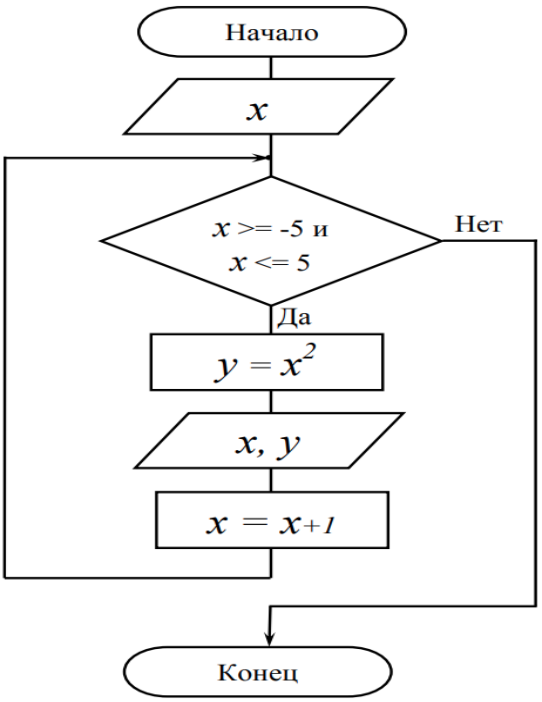


Рисунок 6 – Блок схема к примеру 2

Задания для самостоятельной работы

1. Определить тип вычислительного процесса, реализованного в данном алгоритме (рисунок 7):

а) цикл с постусловием;

б) цикл с предусловием;

в) ветвление;

г) следование;

Чему равен s?

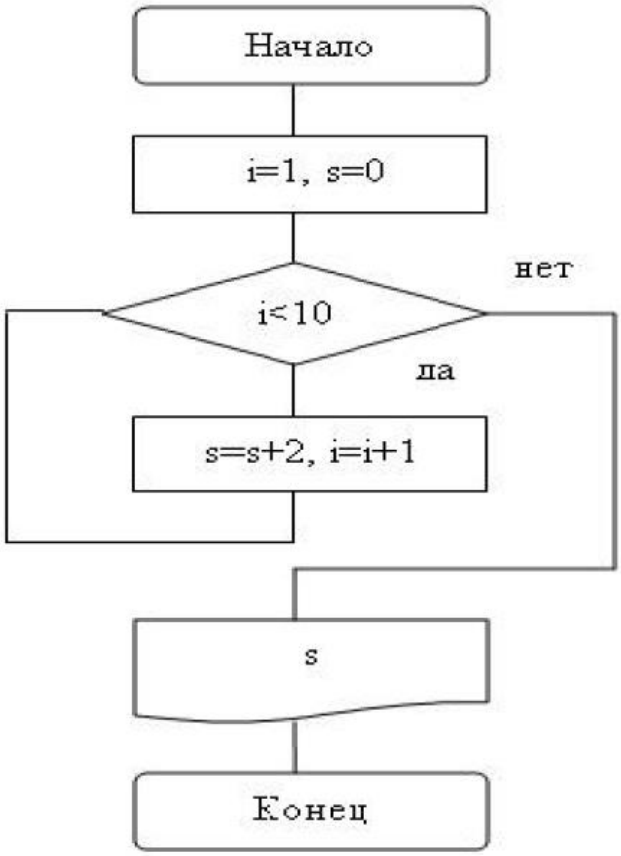


Рисунок 7 – Блок схема к заданию 1

2. В классе N учеников. После контрольной работы было получено: A – пятерок, B – четверок, C – двоек, остальные тройки. Построить блок-схему алгоритма, который определяет процент троек.

3. Придумайте свой собственный циклический алгоритм (например, работа автоматического выключателя, системы трехфазного автоматического повторного включения линии электропередачи) и изобразите в виде блок схемы.

4. Придумать алгоритм, результатом выполнения которого будет вывод последовательности натуральных чисел от 1 до введенного пользователем числа.

5. Напишите алгоритм, который выводит квадраты чисел от 1 до 10.

6. Используя циклы, вывести на экран таблицу умножения на 9.

7. 1 маркер стоит 35 руб. 50 коп. Написать алгоритм вывода на экран таблицу стоимости 2, 3,…20 маркеров.

8. Дано натуральное n. Вычислить: 1 + 1/2 + 1/3 +1/4 + … + 1/n.

9. С клавиатуры вводятся N чисел. Составьте алгоритм, который определяет количество отрицательных, количество положительных и количество нулей среди введенных чисел. Значение N вводится с клавиатуры.

10. Составьте алгоритм, который выводит на экран таблицу умножения числа n на числа от a до b. Числа n, a, b вводятся с клавиатуры.

11. Составьте алгоритм, который выводит на экран таблицу перевода a, a+1, a+2, … b лет в месяцы. Значения a и b вводятся с клавиатуры.

Вопросы для самоконтроля:

1. Дайте определение понятию "Линейный алгоритм".
2. Расположите последовательно действия в алгоритме срабатывания релейной защиты:
   а) измерение параметров сети;
   б) подача команды на отключение;
   в) срабатывание релейной защиты;
   г) отключение сети.
3. Какие блок-схемы соответствуют алгоритму с повторением?
4. Начертите и опишите схемы организации циклических операций (итераций).
5. Что означает термин «параметр цикла», «тело цикла»?
6. Назовите основное принципиальное отличие цикла с параметром от цикла по условию?
7. Напишите и поясните формулу циклического накопления суммы, количества, произведения.

3 ЗНАКОМСТВО С ИНСТРУМЕНТАЛЬНОЙ СРЕДОЙ ПРОГРАММИРОВАНИЯ

Цель работы – познакомиться с языком программирования Pascal.ABC и с интерфейсом среды программирования.

Общие сведения

Среда программирования – это пакет взаимосвязанных файлов, которые позволяют набирать, редактировать, запускать и отлаживать программы.

Информационный блок среды разработки.

После запуска программы открывается окно (рисунок 8).

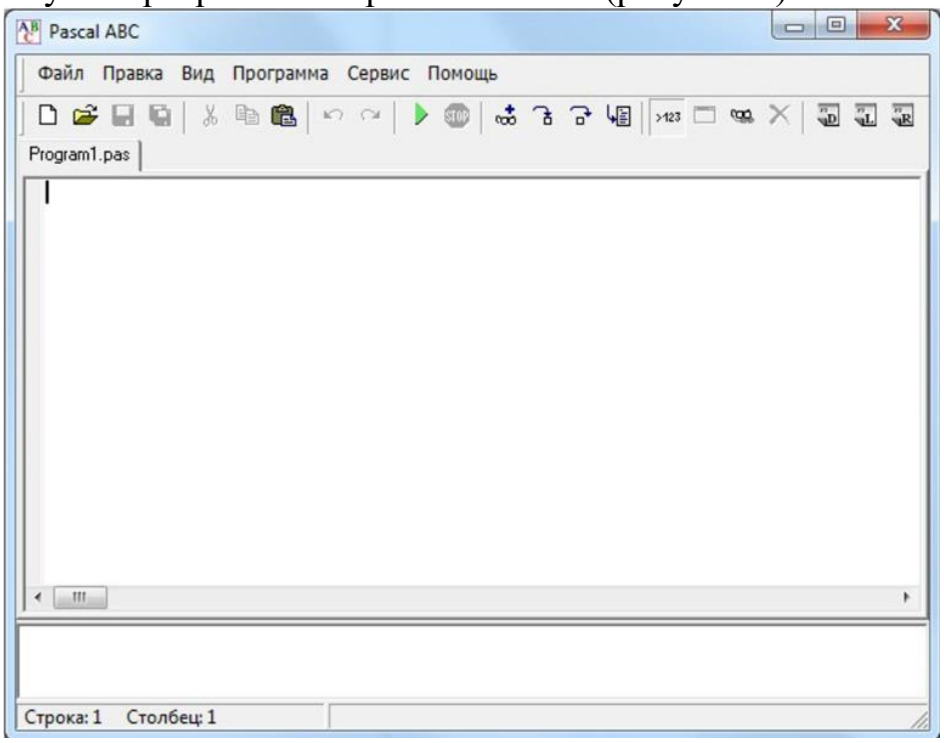


Рисунок 8 – Интерфейс среды разработки Pascal.ABC

Первая строка экрана – меню интегрированной среды, следующая строка – панель инструментов, нижняя строка экрана – строка подсказки и состояния интегрированной среды. Между ними расположено окно редактирования – рабочее поле, в котором можно открывать несколько вкладок для разных программ.

Окно редактирования предназначено для ввода и редактирования текста программы. Место ввода информации обозначено курсором. В верхней левой части окна редактирования выводится служебное имя редактируемого файла, например, Program1.pas.

Знакомство с интерфейсом программы:

1. Найдите строку Меню (сверху) и строку-подсказку (снизу).

2. Поочередно войдите в указанные ниже разделы Меню и найдите следующие команды:

- В меню Файл

  Новый – создать новый фал

  Открыть – открыть файл

Сохранить – сохранить файл
Сохранить как… – сохранить под новым именем
Выход – выйти из среды программирования

- В меню Правка
  Отменить – отменить изменение
  Восстановить – вернуть изменение
- В меню Программа
  Выполнить – выполнить программу
  Остановить – остановить программу.

2. Наберем простейшую программу, соответствующую условию задачи:

Ввести с клавиатуры два целых числа, найти их сумму, результат вывести на экран с поясняющим текстом.

Внимание! Две косые черты (//) отделяют комментарии, их набирать не нужно.

```
program raschet; // название программы
uses crt; // подключаемые модули
var x, y, s:integer; // объявление имен переменных и их типа
begin // начало исполнительной части
writeln('Введите два целых числа'); // написать на экране текст
readln(x,y); // прочитать данные с клавиатуры и запомнить их в переменных
s:=x+y; // выполнить расчет и запомнить его в переменной
writeln('Сумма чисел =',s); // вывести на экран текст и значение переменной
end. // конец программы
```

3. Просмотрите текст файла, обратите внимание на структуру программы.

Структура простейших программ выглядит следующим образом:

```
program ...; заголовок программы и ее имя
var ...; блок объявления переменных и их типа
begin начало исполнительной части программы
...; предложения, обеспечивающие
...; выполнение
...; программы
end. конец программы (точка обязательна)
```

Программа на Pascal составляется из отдельных законченных элементов, называемых предложениями. В Паскале текст программы обычно начинается особым предложением – заголовком следующего вида:

program proba;

где proba – имя текущей программы.

Заголовок необязателен.

В качестве имени программы можно применять комбинацию английских букв и цифр, следует писать в одно слово и нельзя применять служебные слова языка.

Каждое предложение языка должно отделяться от следующего за ним точкой с запятой (;).

Исключение составляют комментарии. Они не отделяются точкой с запятой.

Обычно каждое предложение записывается с новой строки для наглядности и более легкого понимания текста. Для этих же целей используют отступы и выравнивания.

Комментарии предназначены для пояснения задачи и для временного исключения из текста программы некоторых операторов. В тексте они выделяются фигурными скобками { } или отделяются двумя косыми чертами //. Комментарии игнорируются компьютером при выполнении, однако при выводе текста программы – печатаются.

В Паскале имеется особая группа слов, таких как, например: begin, for, end, Program и другие, за которыми закреплены специальные смысловые значения.

Такие слова называются служебными (зарезервированными) и должны употребляться в строгом соответствии с заложенным в них смыслом.

Существует и другая группа имен, имеющих стандартно определенный смысл, например, integer, writeln и другие. Их так и называют – стандартные или предопределенные имена.

Под именем программы располагается ее декларативная часть, здесь компьютеру сообщается обо всех именах констант и переменных, определяемых программистом, и о той роли, которую эти имена должны исполнять в программе.

За декларативной частью следует исполнительная часть программы, обрамляемая словами-ограничителями (логическими скобками): begin и end. Между указанной парой слов и размещаются предложения, выполняющие в программе те или иные действия. Исполнительную часть программы называют телом программы.

4. Запустите набранную программу на выполнение. Если после запуска программы внизу окна появляется красная строчка с сообщением (рисунок 9), то в строке, где находится курсор или в предыдущей (но не всегда, это зависит от ошибки!), внимательно просмотрите всю строчку, найдите и исправьте ошибку. Если сами не справитесь, пригласите преподавателя.

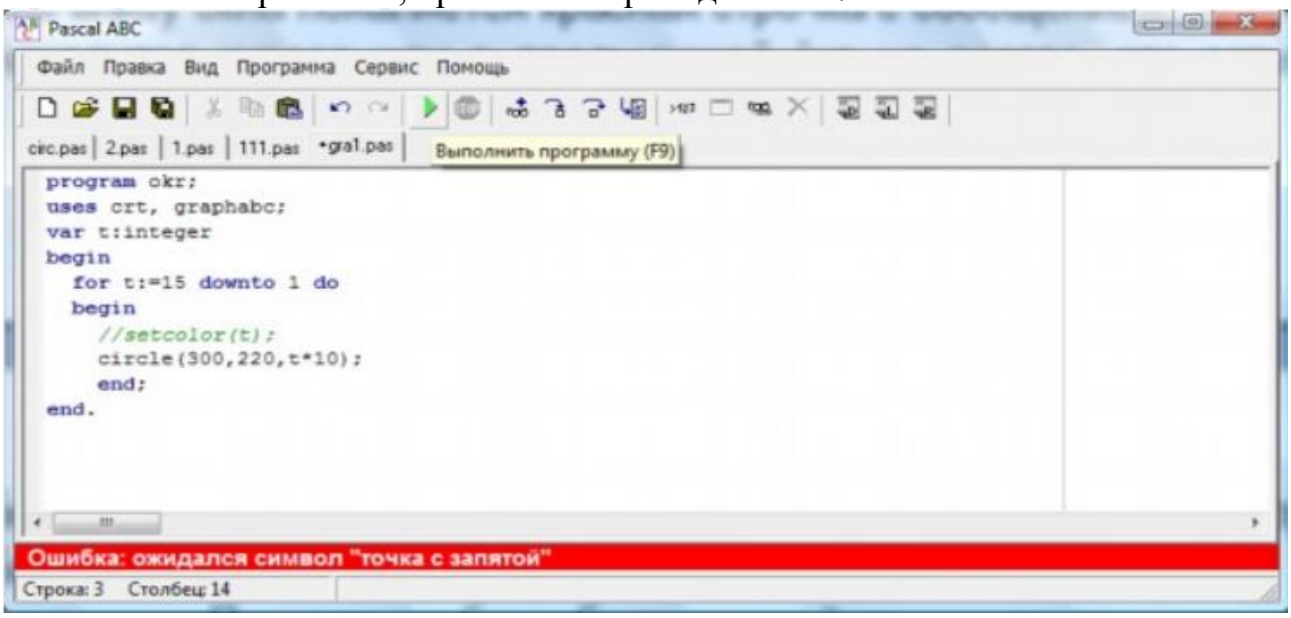


Рисунок 9 – Пример ошибки, обнаруженной Pascal.ABC:

5. После исправления всех ошибок и появления в новом окне начала работы программы, введите нужные данные (если в программе подразумевается ввод нескольких переменных, то это следует делать через Enter или пробел!), получите результат работы и проверьте его на правильность. Так как текст программы и ее работа показываются в разных окнах (если подключен модуль Crt), можно сопоставить программные строки и ее выполнение (рисунок 10).

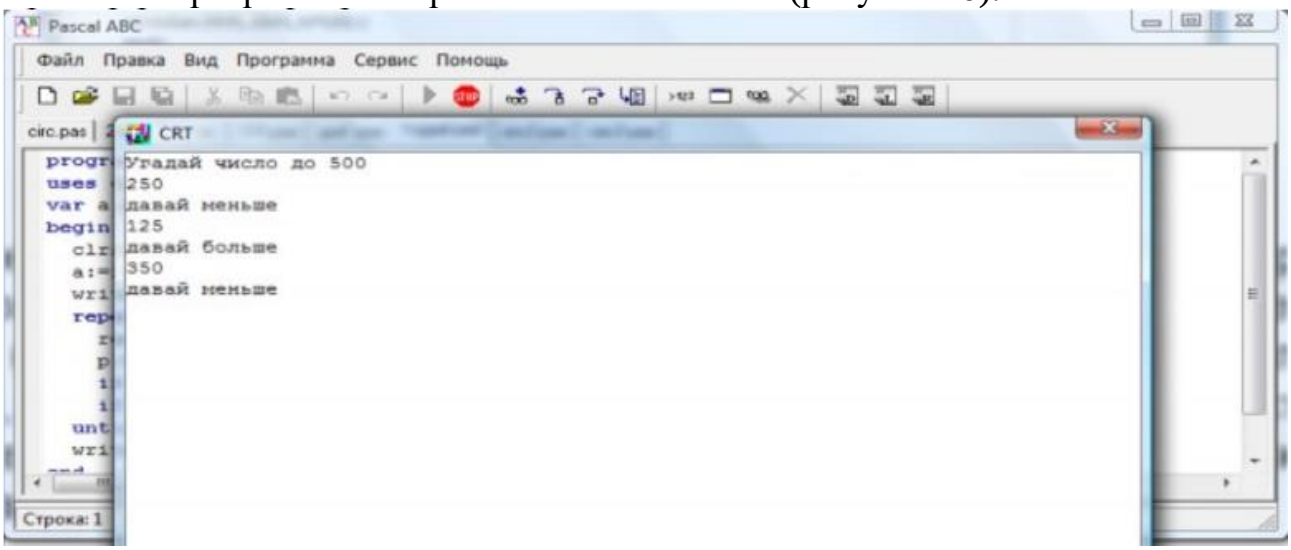


Рисунок 10 – Пример выполнения программы

6. Сохраните набранную программу в своей папке.

7. Разберитесь с работой программы и измените ее так, чтобы она вычисляла не сумму, а разность чисел. Проверьте правильность работы измененной программы. Сохраните программу под новым именем в своей папке.

Вопросы для самоконтроля:

1. Что такое программа?
2. Чем отличается компилятор от интерпретатора?
3. Что такое язык программирования? Синтаксис языка программирования?
4. Как запустить среду PascalABC.NET?
5. Как создать новую страницу? Как открыть текст ранее созданной программы?
6. Как закрыть текущую страницу с текстом программы? Можно ли закрыть все неактивные страницы?
7. Пусть у вас открыто несколько программ. Как сделать активной какую-либо из них?

4 РАЗРАБОТКА ПРОГРАММ ЛИНЕЙНОЙ СТРУКТУРЫ

Цель работы – познакомиться с приёмами работы в среде программирования PascalABC.NET, научиться составлять и тестировать программы для линейных алгоритмов.

Общие сведения

Система программирования Pascal представляет собой единство двух в известной степени самостоятельных начал: компилятора с языка программирования Паскаль и некоторой инструментальной программной оболочки, способствующей повышению эффективности создания программ.

Структура программы, написанной на языке Pascal, состоит из следующих

частей: заголовок программы, раздел описаний и раздел операторов (тело программы), заключенный в операторные скобки begin … end. Операторы в программе могут размещаться как на отдельных строках, так и по несколько в строке. Операторы разделяются точкой с запятой.

Наиболее простым видом организации вычислительного процесса является линейный процесс.

Программа линейного алгоритма представляет собой последовательность операторов, каждый из которых выполняются один раз в порядке его следования.

Основными операторами линейной программы является следующие операторы: оператор присваивания, операторы ввода и вывода информации. Рассмотрим эти операторы.

1 Оператор присваивания

Он имеет следующую структуру:

переменная:=выражение;

Тип переменной и тип выражения должны быть совместимы.

:= – знак присваивания;

Смысл оператора присваивания: вычисляется значение арифметического выражения, представляющего правую часть, а затем вычисленное значение присваивается переменной левой части.

Примеры:

X:=4;

A:=-4. 507;

X1:= A*SIN (X) +2;

2 **Оператор ввода**

Для ввода данных в память ЭВМ в диалоговом режиме, то есть в режиме непосредственного общения человека с машиной используются процедуры Read и Readln.

Общий вид:

Read (переменная 1, переменная 2,…, переменная N);

Readln (переменная 1, переменная 2,…, переменная N);

где переменная – переменная любого типа. Вводимые значения переменных должны отделяться друг от друга пробелом или нажатием клавиши Enter.

3 **Оператор вывода**

Для вывода информации на экран используются стандартные процедуры Write и Writeln.

Общий вид:

Write (выражение 1, выражение 2,…, выражение N);

Writeln (выражение 1, выражение 2,…, выражение N);

где выражение – это то, что нужно вывести на экран. Выражение может быть либо строкой, либо числом, либо выражением, записанным по правилам языка Паскаль.

При выводе информации на экран можно указывать, сколько позиций она будет занимать на экране. Для этого после выражения, значение которого нужно вывести на экран через двоеточие указывают число, задающее количество символов на экране:

Write(выражение: M);

Writeln(выражение: M);

где M – число позиций на экране.

Для вывода на экран значений выражений вещественного типа в дополнении к количеству позиций для всего значения выражения задают и количество позиций для дробной части:

Write(выражение: M:N);

Writeln(выражение: M:N);

где M – число позиций на экране, N – число знаков после запятой.

Задания для самостоятельной работы

1. Написать программу вычисления сопротивления электрической цепи, состоящей из двух параллельно соединенных сопротивлении. Ниже представлен рекомендуемый вид экрана во время работы программы (данные, введенные пользователем, выделенные полужирным шрифтом представлен рекомендуемый вид экрана во время работы программы)

Вычислить сопротивление электрической цепи при параллельном соединении элементов.

Введите исходные данные:

- величина первого сопротивления (ОМ) – 15;
- величина второго сопротивления (ОМ) – 20;
- сопротивление цепи 8.57 ОМ.

2. Написать программу вычисления площади поверхности цилиндра. Ниже представлен рекомендуемый вид экрана во время работы программы.

Вычисление площади поверхности цилиндра. Введите исходные данные:

Радиус основания (см) 5.5

Высота цилиндра (см) 7

Площадь поверхности цилиндра: 431.97 кв.см.

Задание 3. Написать программу вычисления стоимости поездки на автомобиле на дачу (туда и обратно). Исходными данными являются: расстояние до дачи (в километрах); кол-во бензина, которое потребляет автомобиль за 100 км пробега; цена одного литра бензина.

Рекомендуемый вид экрана во время работы программы:

- вычисление стоимости поездки на дачу и обратно;
- расстояние до дачи (км) 67;
- расход бензина (литров на 100 км пробега) 8.5;
- цена 1 литра бензина (руб.) 6.5;
- поездка на дачу и обратно обойдется в 74.04 руб.

3. Даны два числа. Найти среднее арифметическое квадратов этих чисел и среднее геометрическое модулей этих чисел.

4. Вычислить расстояние между двумя точками с данными координатами (x1, y1) и (x2, y2).

5. Дано двухзначное число, определить сумму его цифр.

6. Дано время в минутах, вывести на экран время в формате часы : минуты.

Вопросы для самоконтроля:

1. С какого служебного слова начинается раздел исполнения операторов?
2. Какие операторы могут быть использованы в линейной программе?
3. Если в программе есть оператор x:=1.2, то какой тип должна иметь переменная x?
4. Каким образом будет располагаться текст на экране после выполнения следующего фрагмента?

```
Write (‘С ’);
Writeln (‘наступающим новым ’);
Write (‘годом’);
```

5. Какие значения будут выведены на экран после выполнения следующего фрагмента программы?

```
x:=4; y:=5;
y:=y+Sqrt(x);
Writeln(‘y=’, y:2:0, ‘x=’, x:2:0)
```

5 РАЗРАБОТКА ПРОГРАММ РАЗВЕТВЛЯЮЩЕЙСЯ СТРУКТУРЫ

Цель работы – приобретение навыков использования для организации ветвления условного оператора в языке программирования Pascal.

Общие сведения

Условный оператор

Условие может быть простым и составным и может принимать два значения – True и False.

В условном операторе знак «;» ставится только в его конце, перед разделителями Then и Else он не допустим.

Условный оператор может быть вложенным. При этом разделитель Else принадлежит ближайшему сверху оператору If. Например:

```
If условие1
   Then оператор1
   Else
      If условие2
      Then оператор2
      Else
        If условие3
        Then оператор3;
```

Здесь оператор3 выполнится в случае, если ложно условие1, ложно условие2 и истинно условие3.

Если после разделителей Then или Else необходимо записать составной оператор, его заключают в операторные скобки Begin…End. Например:

```
If условие
   Then
   Begin
     оператор1
     …
     операторN
  End
Else
  Begin
  …
  End;
```

Задания для самостоятельной работы

1 Написать программу вычисления стоимости покупки с учётом скидки. Скидка в 10% предоставляется, если сумма покупки больше 1000 рублей. Алгоритм решения задачи: с клавиатуры запросить стоимость покупки. Если введённое значение больше 1000, то уменьшить её на 10 процентов, иначе сумму оставить без изменений.

2. Заданы площади круга и квадрата. Определить, поместится ли квадрат в круге.

3. Даны две точки A(x1,y1) и B(x2,y2). Определить какая из точек находится

ближе к началу координат.

4. Рассчитать надбавку к зарплате за стаж, если стаж от 2 до 5 лет, надбавка составляет 2%, если стаж от 5 до 10 лет - 5%. Ввести с клавиатуры зарплату и стаж, вывести надбавку и сумму к выплате.

5 Дано натуральное число. Определить, является ли оно четным, или оканчивается цифрой 3.

6. Ввести с клавиатуры координаты точки A(x,y). Определить, лежит ли данная точка в первой четверти. Ответ вывести в виде сообщения.

7. Известны две скорости, одна в километрах в час, другая в метрах в секунду. Какая из них больше?

8. Ввести с клавиатуры значения трех сторон треугольника a, b и c и определить, является ли он равнобедренным. Ответ вывести в виде сообщения.

Вопросы для самоконтроля:

1. Что необходимо предусмотреть для использования составного оператора внутри оператора If?

2. Приведите пример оператора выбора с использованием составного оператора внутри.

3. Назовите особенности использования разделителя Else для оператора If?

4. В каком месте программы может находиться оператор Begin, а где не может?

6 РАЗРАБОТКА ПРОГРАММ С ИСПОЛЬЗОВАНИЕМ ЦИКЛА

Цель работы – является получение навыков в составлении циклических программ с использованием оператора цикла for.

Общие сведения

Цикл – это многократно повторяющаяся последовательность операторов.

В Паскале используются 3 типа циклов:

- цикл с постусловием (Repeat);
- цикл с предусловием (While);
- цикл с параметром (For).

1 Оператор цикла For

Оператор цикла For используется когда заранее известно количество повторений цикла и когда шагом цикла является целое число.

Синтаксис оператора цикла с параметром:

For <параметр цикла> := <начальное значение> To/DownTo <конечное значение> Do <оператор>;

<Оператор> выполняется до тех пор, пока <параметр цикла> изменяет свое значение от <начального значения> до <конечного значения>.

Например,

For I:=1 To 10 Do x:=x+2;

В этом случае параметр цикла каждый раз увеличивается на 1 автоматически.

For I:=10 DownTo 1 Do x:=x+2;

В этом случае параметр цикла каждый раз уменьшается на 1 автоматически.

ВНИМАНИЕ! В теле цикла НЕ желательно изменять параметр цикла (для избежания зацикливания). Например:

For I:=1 To 10 Do I:=I+2; {последний раз цикл должен выполниться для I равного 10 и если каким-то образом переменная I «проскочит» это значение, то произойдет зацикливание программы}

Если все-таки программа зациклилась, воспользуйтесь комбинацией клавиш Ctrl+Break для возврата в среду программирования Паскаль.

Если телом цикла For является составной оператор, то его заключают в операторные скобки Begin …end. Например:

```
For I:=1 to 10 Do
Begin
оператор1;
…
операторN;
end;
```

Перед началом цикла параметру цикла присваивается <начальное значение>. И если до этого цикла переменная, являющаяся параметром цикла хранила какое-то значение, оно теряется.

2 Процедуры Break и Continue

Процедура Break предназначена для принудительного досрочного выхода

из цикла. Как правило, используется для уменьшения времени выполнения программы (время выполнения программы можно сократить за счет невыполнения некоторых операторов «в холостую», это когда результат выполнения таких операторов ни на что не влияет). Например:

```
For I := 1 To 100 Do
Begin
Readln(k);
If k mod 2 <> 0 Then Break; {принудительный выход из цикла если введено нечетное число k}
If k>0 Then s:=s+k ;
 End;
```

Если после процедуры Readln(k) будет введено нечетное значение, цикл прекратит свое выполнение и далее будет выполнен оператор, следующий за оператором End. В данном примере процедура Break позволяет сократить время выполнения программы, по меньшей мере, на время выполнения оператора If k>0 (т.е. на время проверки условия k>0).

Процедура Continue принудительно вызывает начало новой итерации (повторения) цикла, даже если предыдущая не закончена. Например:

```
For I := 1 To 100 Do
   Begin
      Readln(k);
      If k = 0 Then Continue; {вызов начала выполнения тела цикла, начиная
         с процедуры ввода Readln(k)}
      If k>0 Then s:=s+1;
      If k<0 Then d:=d+1;
  End;
```

В данном примере если с помощью процедуры ввода Readln(k) в переменной k окажется ноль, то два последних условных оператора будут проигнорированы и запрошено новое значение переменной k (тело цикла начнет выполняться сначала). Т.е. процедура Continue также позволяет экономить время выполнения программы. В принципе условного оператора If k = 0 Then Continue может не быть. Но тогда в случае нулевого значения переменной k будет зря тратиться время на проверку двух последних условий.

Задания для самостоятельной работы

1 Написать программу, выводящую на экран заглавные буквы латинского алфавита в обратном порядке в одну строку через запятую.

В качестве параметра цикла for может быть переменная не только целого типа, но и символьного. Например,

```
Var C:char;
Begin
For C:='a' to 'k' do writeln(C); {параметр цикла C перебирает значения
букв латинского алфавита от a до k по порядку}
```

В данной задаче необходимо вывести буквы в обратном порядке, т.е. не по алфавиту. Значит, параметр цикла должен не увеличиваться, а уменьшаться, следовательно, вместо разделителя To необходимо использовать разделитель DownTo.

2 Вывести на экран все нечетные числа из последовательности чисел, вводимых с клавиатуры. Количество чисел в исходной последовательности N описать как константу в разделе описания констант Const. Например, дана последовательность чисел: 2 5 12 36 45 7 9 22. Результатом работы программы будет следующая последовательность чисел: 5 45 7 9.

3. По блок-схеме алгоритма (рисунок 11), приведенной на рисунке, составить программу решения задачи и заполнить таблицу 2.

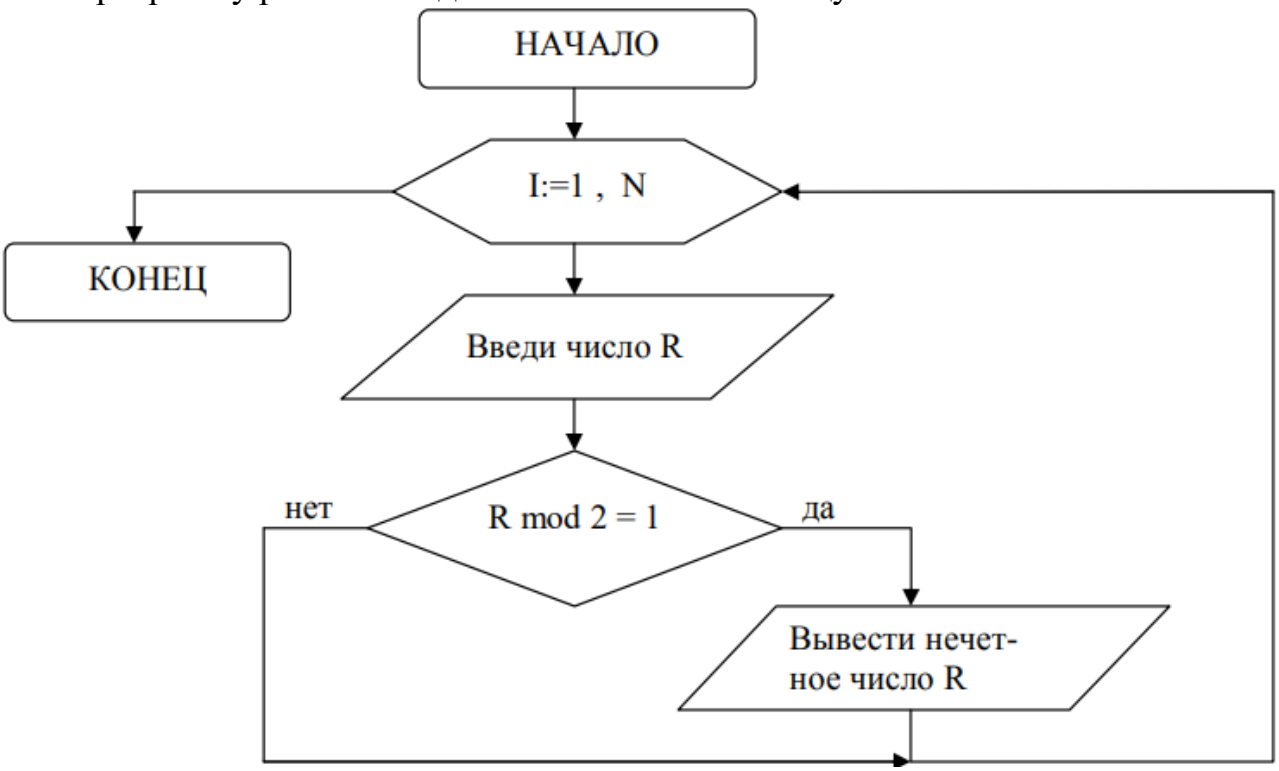


Рисунок 11 – Блок схема алгоритма к заданию 3

Таблица 2 – Результаты выполнения программы

Вводимая последовательность R	Результат

4. Написать программу, вычисляющую сумму и среднее арифметическое последовательности положительных чисел, которые вводятся с клавиатуры.

5. Написать программу, которая вычисляет среднюю (за неделю) температуру воздуха. Исходные данные должны вводиться во время работы программы.

6. Составить программу, вычисляющую сумму квадратов чисел от 1 до введенного вами целого числа n.

7. Начав тренировки, спортсмен в первый день пробежал 10 км. Каждый день он увеличивал дневную норму на 10% нормы предыдущего дня. Какой суммарный путь пробежит спортсмен за 7 дней.

8. Написать программу, которая выводит таблицу степеней двойки (от нулевой до десятой).

Вопросы для самоконтроля:

1. В каких случаях может быть использован оператор цикла for?
2. Какие требования предъявляются к параметру цикла for?
3. Как изменится цикл for, если параметр цикла должен не увеличиваться, а уменьшаться с каждой следующей итерацией цикла.
4. Если телом цикла является составной оператор, что при этом изменится для цикла for?
5. С помощью какой процедуры можно прервать выполнение цикла? Приведите пример.
6. Как можно возобновить выполнение цикла, если еще не завершен предыдущий проход цикла? Приведите пример.
7. По приведенному фрагменту программы определить сколько раз на экране появится слово «цикл».

7 ОБРАБОТКА ОДНОМЕРНЫХ ЧИСЛОВЫХ МАССИВОВ. СОРТИРОВКА ОДНОМЕРНЫХ МАССИВОВ

Цель работы – знакомство с основными приемами работы с одномерными массивами и алгоритмом сортировки одномерного массива.

Общие сведения

Массив – множество данных. Структура информации, которая хранит в себе набор значений (элементы), идентифицируемых при помощи так называемого индекса (или их набора), принимающего целые (приводимые к целым) значения из заданного непрерывного интервала.

Массивом называют упорядоченный набор элементов, каждый из которых содержит в себе то или иное значение. Идентификация соответствующего компонента производится с помощью одного или нескольких индексов.

Самый простой массив:

- имеет постоянную длину;
- содержит единицы данных одного и того же типа;
- предусматривает в виде index целые числа.

Массивы бывают:

- одномерными (в них всего один индекс);
- двумерными (с двумя индексами);
- трехмерными и так далее.

Одномерный вариант (таблица 3) нестрого соответствует вектору в математике, двумерный – матрице. Соответствующие компоненты на практике встречаются чаще остальных. Работа с трехмерными и более крупными множествами информации – огромная редкость.

Массивы бывают обычными. Их называют статическими или динамическими. У них «размерность» не меняется по мере обработки программного кода. Есть динамические множества. В них размер корректируется в зависимости от выполненного приложения или задачи.

Таблица 3 – Типовые действия с одномерными массивами

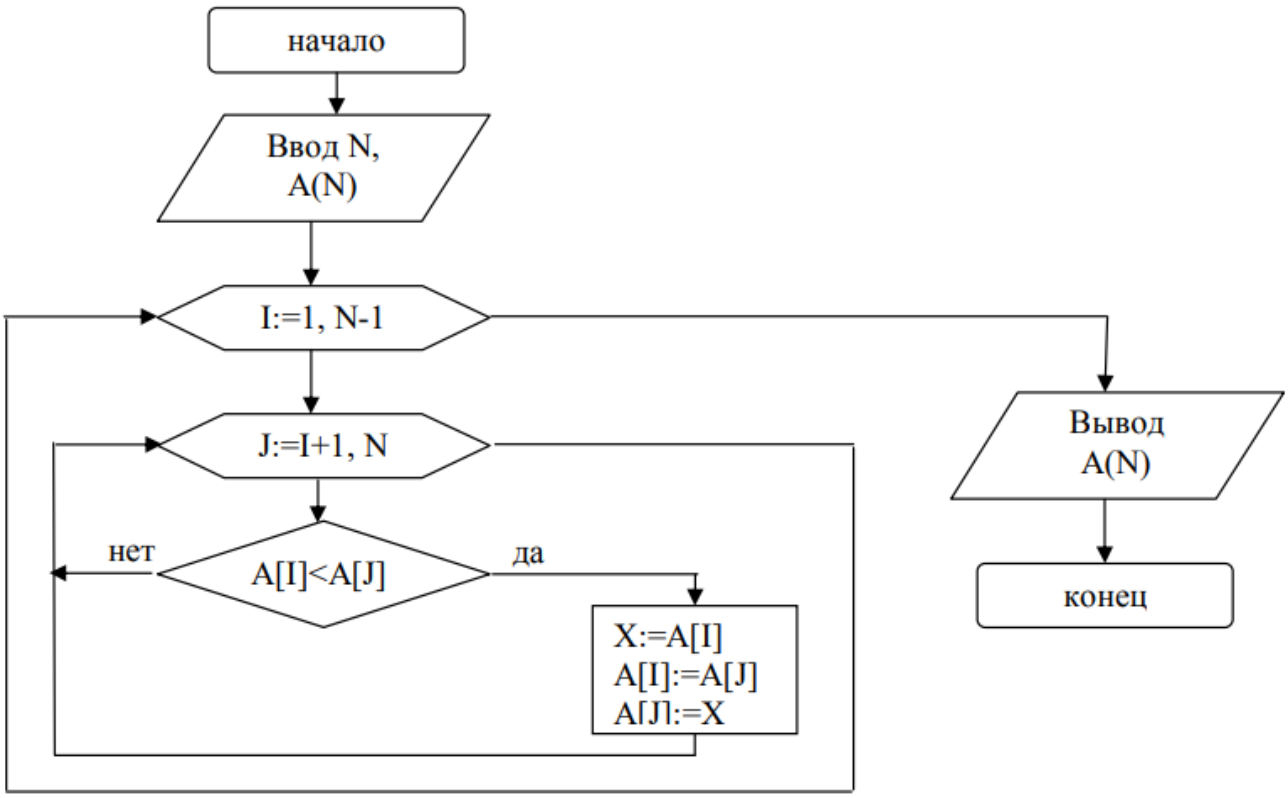
Наименование	Примеры использования
Описание одномерного массива, здесь n - количество элементов в массиве, mas – тип, обозначающий одномерный массив из 5 элементов целого типа, A – переменная типа mas, т.е. переменная типа одномерный массив из 5 элементов целого типа	Const n=5; Type mas=array[1..n] of integer; Var A:mas;
Ввод элементов одномерного массива с клавиатуры	For I:=1 to n do Read(A[I]);
Вывод элементов одномерного массива на экран	For I:=1 to n do write(A[I],’ ‘); Writeln;
Заполнение элементов одномерного массива с помощью генератора случайных чисел	For I:=1 to n do Begin

	A[I]:=Random(50); write(A[I],’ ‘); End; Writeln;
Преобразование каждого элемента массива, т.е. изменение значения каждого элемента массив	For I:=1 to n do A[I]:= A[I]-6; {уменьшает каждый элемент массива на 6}
Определение суммы элементов одномерного массива	S:=0; For I:=1 to n do S:=S+A[I];
Определение количества элементов одномерного массива, удовлетворяющих некоторому условию	K:=0; For I:=1 to n do If A[I] > 4 Then K:=K+1;

Определение экстремума (максимального или минимального элемента массива)	Min:=A[1]; K:=1;
	For I:=2 to n do
	If A[I] < Min Then
	Begin
	Min:= A[I]; K:=I;
	End;
	Writeln(‘мин. элемент = ’,A[k]);

Задания для самостоятельной работы

1. По предложенной блок-схеме написать программу для следующего задания: отсортировать элементы одномерного массива по убыванию.



2. Заполнить одномерный массив с помощью генератора случайных чисел, среднее арифметическое элементов массива.

Алгоритм решения задачи состоит из четырех действий:

1) Заполнить массив, используя генератор случайных чисел;

2) Определить сумму элементов массива;

3) Определить среднее арифметическое по формуле SR:=S/N, где S – сумма элементов, N – константа, обозначающая количество элементов в массиве.

4) Вывести результат.

3. Изменить программу, составленную в задании 1 так, чтобы вычислялось среднее арифметическое отрицательных элементов массива

В этом случае необходимо определить сумму только тех элементов массива, которые удовлетворяют условию A[i]<0. А также добавить новую переменную для количества отрицательных чисел. Т.е., если выполняется условие A[i]<0, необходимо выполнить два действия – добавить элемент к сумме (S:=S+A[i]) и увеличить количество элементов на 1 (K:=K+1). Получается, что в случае выполнения условия необходимо выполнить два действия, поэтому их требуется заключить в операторы Begin End. Постройте блок-схему алгоритма и напишите программу.

4. Задана последовательность из N вещественных чисел. Определить, сколько среди них чисел меньших К, равных К и больших К.

5. Все элементы одномерного массива, находящиеся перед минимальным элементом заменить нулями.

6. Обнулить элементы одномерного массива, находящиеся перед минимальным элементом.

7. Дана последовательность действительных чисел a1, a2, …, an. Выяснить, будет ли она возрастающей.

8. Задан массив действительных чисел. Определить, сколько раз меняется знак в данной последовательности чисел, заполнить номера позиций, в которых происходит смена знака.

Вопросы для самоконтроля:

1. Продолжите определение. Массив – это тип данных, представляющий собой…

2. Напишите составное условие, проверяющее элемент массива A[i] на нечетность и положительность.

3. Что означает следующий фрагмент программы, где A(n) – одномерный массив? For I:=1 to n do A[I]:=0

4. Напишите фрагмент программы, определяющий произведение элементов массива.

8 РАЗРАБОТКА ПРОГРАММ С ИСПОЛЬЗОВАНИЕМ ДВУМЕРНЫХ МАССИВОВ. СОРТИРОВКА ДВУМЕРНЫХ МАССИВОВ

Цель работы – познакомиться с приемами работы и сортировки с двумерных массивов.

Общие сведения

Двумерные массивы (таблица 4) на языке программирования Pascal описываются так:

array [1..m, 1..n] of базовый_тип

Однако можно их описывать как массив массивов:

array [1..m] of array [1..n] of базовый_тип

Рассмотрим простой пример работы с двумерным массивом. Сначала заполним его данными, а затем выведем их на экран в виде таблицы.

```
var
   matrix: array[1..3,1..5] of integer;
   i, j: integer;

begin
   writeln('Введите 15 чисел: ');

   for i := 1 to 3 do
      for j := 1 to 5 do
         read(matrix[i,j]);

   for i := 1 to 3 do begin
      for j := 1 to 5 do
         write(matrix[i,j], ' ');
      writeln
   end;

end.
```

Размерность массива (т.е. количество содержащихся в нем значений) определяется произведением количества строк на количество столбцов. В примере выше в массив помещается 15 значений.

Таблица 4 – Типовые действия с двумерными массивами

Наименование	Примеры использования
Описание двумерного массива, здесь n-количество строк, m – количество столбцов, mas – тип, обозначающий двумерный массив 5 x 6, A – переменная типа mas, т.е. переменная типа двумерный массив из 5 строк и 6 столбцов	Const n=5, m=6; Type mas=array[1..n, 1..m] of integer; Var A:mas;

Ввод элементов двумерного массива с клавиатуры	For I:=1 to n do For J:=1 to m do Read(A[I, J]);

Вывод элементов двумерного массива на экран	For I:=1 to n do
	  Begin
	  For J:=1 to m do write(A[I, J],' ');
	  Writeln;
	  End;

Заполнение элементов двумерного массива с помощью генератора случайных чисел	For I:=1 to n do
	  Begin
	  For J:=1 to m do
	  Begin
	  A[I, J]:=Random(50);
	  write(A[I, J],' ');
	  End;
	  Writeln;
	  End;

Преобразование элементов массива, т.е. изменение значений элементов массива	For I:=1 to n do For J:=1 to m do A[I, J]:= A[I, J]*2;
Определение суммы элементов двумерного массива	S:=0; For I:=1 to n do For J:=1 to m do S:=S+A[I, J];

Определение количества элементов двумерного массива, удовлетворяющих некоторому условию	K:=0;
	For I:=1 to n do
	For J:=1 to m do
	If A[I, J] > 4 Then K:=K+1;

Определение экстремума (максимального или минимального элемента массива)	Min:=A[1,1];
	For I:=1 to n do
	For J:=1 to m do
	If A[I, J] < Min Then Min:= A[I, J];

Задания для самостоятельной работы

1. Заполнить двумерный массив с помощью генератора случайных чисел, определить, сколько процентов от общего числа элементов массива составляют нулевые элементы. Проценты можно подсчитать по формуле P:=k/(n*m)*100, где k – количество нулевых элементов, n*m – общее количество элементов массива из n строк и m столбцов. Но перед этим, чтобы определить k необходимо каждый элемент массива A[i,j] проверить на равенство нулю.

2. Заполните константный массив 5x5 числами 1…5. Выведите на экран этот массив, а также транспонированный массив, т.е. такой, значения строк которого являются значениями столбцов первоначального массива.

3. Найти сумму элементов столбца и строки массива, на пересечении которых находится максимальный элемент.

4. Заполнить массив nxn значениями 1…n2 по спирали, начиная с верхнего

левого угла.

5. Отсортировать каждую строку двумерного массива по возрастанию.

6. Отсортировать каждый столбец двумерного массива по убыванию.

7. Отсортировать главную диагональ квадратной матрицы по возрастанию.

8. Обнулить элементы главной диагонали. Алгоритм решения задачи состоит из трех действий:

1) ввести массив;

2) обнулить элементы, расположенные на главной диагонали;

3) вывести массив.

Пример работы программы

Исходный массив	**Результат**
1 1 1 1	0 1 1 1
2 2 2 2	2 0 2 2
3 3 3 3	3 3 0 3
4 4 4 4	4 4 4 0

9. В каждом столбце прямоугольной матрицы поменять местами первый и последний элементы.

10. Удвоить наименьший элемент на главной диагонали квадратной матрицы.

11. Дана прямоугольная матрица, создать одномерный массив, каждый элемент которого – сумма соответствующей строки матрицы.

Вопросы для самоконтроля:

1. Приведите пример, как можно использовать генератор случайных чисел для получения дробных отрицательных чисел.

2. Как определить, что элемент двумерного массива находится над побочной диагональю? под главной диагональю?

3. Что означает следующий фрагмент программы, где A(N, M) – двумерный массив?

```
For I:=1 to n do A[I, 5]:=0;
```

4. В следующем фрагменте программы определяется количество нулевых элементов двумерного массива A(N, M) в каждой строке. Что необходимо изменить, чтобы это количество определялось для каждого столбца? Запишите в тетрадь указанный фрагмент программы с учетом изменений.

```
For I:=1 to n do
  begin
    K:=0;
    For J:=1 to m do If A[I,J]=0 Then k:=k+1;
    Writeln('кол-во нулевых элементов в ',i,' строке равно',k);
  End;
```

9 ОБРАБОТКА СТРОК. ФУНКЦИИ И ПРОЦЕДУРЫ ДЛЯ РАБОТЫ СО СТРОКАМИ

Цель работы – научиться использовать стандартные процедуры и функции для строк.

Общие сведения

Строка – это последовательность символов (таблица 5). Количество символов в строке может динамически меняться в пределах от 0 до 255.

1. **Описание переменных строкового типа**:

a. В разделе описания переменных VAR

VAR

St: string [12]; -объявлена строка с длиной 12 символов

St1:string; - по умолчанию длина строки =255

b. В разделе констант:

CONST St=’Сегодня наступила весна’;

c. В разделе описания типов:

TYPE Stroka=string[8];

VAR St:stroke;

2. К отдельным символам строки можно обратиться по номеру индексу данного символа в строке в квадратных скобках.

St[1]- первый символ строки st;

St[4]- четвертый символ строки st;

St[0]- специальный символ- указатель на длину строки.

3. Строковые процедуры и функции

4. Стандартные функции, определенные для символьного типа Char.

Таблица 5 – Процедуры и функции работы со строками

Наименование	Примеры использования
Функция Length – возвращает целое число – текущую длину строки	A:=Length(s); For I:=1 to Length(Slovo) do …
Функция Pos – возвращает номер (целое число) первого символа строки S1 при его первом появлении в строке S2	S1:=’это’; S2:=’Найди это слово’; A:=Pos(S1, S2); {переменная A получит значение - 7}
Функция Copy – копирует N символов строки S, начиная с позиции P, т.е. возвращает строку	P:=4; N:=3; S:=’компьютер’; S1:=Copy(S, P, N); {переменная S1 получит значение ‘пью’, строка S останется без изменений}
Процедура Delete – удаляет N символов строки S, начиная с позиции P	S:=’параметр’; P:=1; N:=4; Delete(S, P, N); Writeln (S); {на экране появится слово ‘метр’, при этом длина строки S равна 4};

Процедура Insert – вставляет строку S1 в строку S2, начиная с позиции P	S1:=’это ’; S2:=’Найди слово’; P:=7; Insert(S1, S2, P); Writeln(S1); {на экран выведутся 4 символа ‘это ’} Writeln(S2); {на экране появится ’Найди это слово’, если позволяет длина строки}
Процедура Str – преобразует число X в строку S	Var Y: integer; X:real; BEGIN Y:=1999; Str(Y:6, S); Writeln (s); {на экране 6 символов: __1999}
Процедура Val – преобразует строку S в целое или дробное число X. Если преобразование возможно (т.е. если в строке все символы являются символами числа), то в переменную Z будет записан ноль, иначе там будет номер первого символа строки, который не может быть символом числа	Var X: real; S: String[5]; Z: integer; BEGIN S:=’17.25’; Val(S, X, Z); Writeln (‘X=’,X:5:2,’ ошибка=’, Z); {на экране появится: X=17.25 ошибка=0} END.

Так как строка – это одномерный массив элементов символьного типа Char, значит к ним применимы процедуры и функции, определенные для символьного типа данных.

Chr(x)- возвращает значение символа по его коду ASCII.

Ord(Ch)- возвращает код заданного символа ch.

Pred(ch)- возвращает предыдущий символ.

Succ(ch)- возвращает следующий символ.

Например:

Chr(66)=’B’;

Ord(‘A’)=65;

Pred(‘B’)=’A’;

Succ(‘B’)=’C’

5. **Основные операции над строками**

a. Введение строки с клавиатуры:

```
Var st:string;
Begin
 Writeln(‘введите строку’);
Readln(st);
…
```

b. Задание строки и вывод строки на экран:

```
Var
St:string;
Begin
 St:=’Привет!’;
 Writeln(st);
End.
```

с. Подсчет количества конкретного символа в строке:

```
k:=0;
for i:=1 to length(st) do
 if st[i]=’s’ then k:=k+1;
```

d. Замена пробела в строке на символ ‘*’

```
 for i:=1 to length(st) do
 if st[i]=’ ’ then st[i]:=’*’;
```

е. Обращение введенного слова, т.е. если задана строка ‘Маша’, то программа воэвращает ‘ашаМ’

```
Var
 i,n:integer;
 s:=string;
 ch:char;
Begin
 Write(‘введите строку=’); readln(s);
 n:=length(s);
 for i:=1 to trunc(n/2) do
 begin
 ch:=s[i];
 s[i]:=s[n-i+1];
 s[n-i+1]:=ch;
 end;
 writeln(s);
End.
```

Задания для самостоятельной работы

1. Напишите программу, которая вводит строку и выводит ее, сокращая каждый раз на 1 символ до тех пор, пока в строке не останется 1 символ. Пример работы программы:

Введите строку:
Abcde
Результат:
Abcd
Abc
Ab
A.

2. Введите 2 целых числа. Преобразуйте числа в 2 строки, объедините их в одну и выведите результат.

3. Напишите программу, определяющую число слов в строке. Одно слово от другого отделяется 1 пробелом. Пример работы программы:

Введите строку:

Делу время, а потехе час.

Количество слов в строке: 5

4. Вставьте в строку, заданную с клавиатуры 2 пробела, начиная с позиции, заданной с клавиатуры. Если введенный номер позиции превышает длину строки программа должна выдавать ошибку. Примеры работы программы:

а) Введите строку: ПРИВЕТ

Введите номер позиции: 4

Результат: ПРИ ВЕТ.

б) Введите строку: ПРИВЕТ

Введите номер позиции: 14

Результат: Ошибка!

Вопросы для самоконтроля:

1. Что представляет собой строка?
2. Какая максимальная длина может быть у строки?
3. Как можно обратиться к каждому символу строки?
4. Что обозначает st[0]?
5. Какие функции предназначены для преобразования типов? Опишите их.

10 ПРОЦЕДУРЫ И ФУНКЦИИ

Цель работы – научится применять процедуры и функции при решении задач, использующих часто повторяющиеся операторы; уметь правильно описывать процедуры и функции и обращаться к ним.

Общие сведения

Любая подпрограмма начинается с заголовка, который является обязательным. Для процедуры он состоит из служебного слова Procedure с последующим именем процедуры и, если необходимо, списком формальных параметров в круглых скобках.

Для вызова процедуры из основной программы или другой подпрограммы

необходимо записать оператор, состоящий из имени процедуры и списка фактических параметров, которые должны совпадать по количеству и типам с

формальными параметрами процедуры, например:

Пример 1. Использование глобальных

переменных

```
Uses crt;
var a,b,c:integer;
procedure summa;
   begin
     c:=a+b;
   end;
BEGIN
   write('a='); readln(a);
   write('b='); readln(b);
   summa;
   writeln('сумма=',c);
END.
```

Для объявления функции используется служебное слово Function с последующим именем функции и, если необходимо, списком формальных параметров в круглых скобках, затем через двоеточие записывается тип функции – тип возвращаемого параметра. Функция может возвращать параметры целого, вещественного, символьного, логического, строкового типа.

За заголовком подпрограммы могут идти такие же разделы, что и в основной программе, заканчивается подпрограмма точкой с запятой. Необходимо отметить, что в теле функции хотя бы один раз имени функции должно быть присвоено значение. Результатом выполнения функции будет последнее присваиваемое значение. Если такого оператора присваивания нет или он не выполняется, то возвращаемое функцией значение будет неопределенным.

Для вызова функции из основной программы или другой подпрограммы следует в выражении, где необходимо использовать значение функции, указать имя функции со списком фактических параметров, которые должны совпадать по количеству и типам с формальными параметрами функции, например:

Пример 2. Функция вычисления факториала числа

```
var x:byte;
function factor(n:byte):LongInt;
var i:byte; f:LongInt;
begin
  f:=1;
  for i:=2 to n do f:=f*i;
  factor:=f;
end;
BEGIN
  Write('x='); Readln(x);
  Writeln('Факториал числа=',factor(x));
  Readln;
END.
```

Задания для самостоятельной работы

1. Написать процедуру удаления из строки всех пробелов.
2. Написать процедуру, выводящую на экран фразу «Добрый день!» заданное количество раз.
3. Написать процедуру, обнуляющую элементы главной диагонали двумерного массива.
4. Написать процедуру, добавляющую в конец строки столько восклицательных знаков «!», сколько символов в строке.
5. Написать процедуру, удаляющую из строки первое слово.
6. Написать процедуру, удаляющую из строки последнее слово.
7. Написать процедуру, выводящую на экран элементы одномерного массива через запятую и в обратном порядке.
8. Написать процедуру, выводящую на экран элементы побочной диагонали.
9. Написать функцию для определения максимального элемента одномерного массива.
10. Написать функцию для определения минимального элемента двумерного массива.
11. Написать функцию для определения суммы положительных элементов одномерного массива.
12. Написать функцию для определения количества элементов одномерного массива, кратных k.
13. Написать функцию для определения произведения четных элементов одномерного массива.
14. Написать функцию для определения суммы элементов главной диагонали матрицы.
15. Написать функцию для определения количества гласных букв в строке.
16. Написать функцию, которая определяет сколько процентов от общего числа символов составляют согласные буквы.

17. Написать функцию, которая определяет сколько процентов от общего числа элементов одномерного массива составляют нулевые элементы.

18. Заданы два одномерных массива разного размера. Организовать три процедуры: ввод элементов массива, вывод элементов массива и процедуру увеличения каждого элемента массива в 3 раза.

Вопросы для самоконтроля:

1. Чем отличается структура подпрограммы от структуры программы?

2. Что такое формальный параметр подпрограммы? Укажите, какие формальные параметры, для каких процедур вы использовали в задании 18.

3. Что такое фактический параметр подпрограммы? Укажите, какие фактические параметры вы использовали в задании 18.

4. В чем состоит отличие при использовании в описании подпрограммы параметра-переменной и параметра-значения?

5. Какие данные в задании 18 относятся к глобальным данным, а какие к локальным данным?

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Галиев, Г.А. Основы алгоритмизации и программирование: учебно - метод. пособие. – Краснодар: КубГАУ, 2013. – 94 с.

2. Долинер, Л. И. Основы программирования в среде PascalABC.NET: учебное пособие / Л. И. Долинер. – Екатеринбург: Изд-во Урал. ун-та, 2014. – 128 с

3. Лобачева М.Е. Основы алгоритмизации и программирования. Учебное пособие. – Самара: КС ПГУТИ, 2019. – 160с.

4. Соловьева Т. Н. Информатика. Основы алгоритмизации и программирования: учеб. пособие / Т. Н. Соловьева: Перм. гос. нац. исслед. ун-т. – Пермь, 2018. – 118 с.

5. Федоров В. Б. Основы алгоритмизации и программирования. – Москва: МИРЭА - Российский технологический университет, 2022. – 203 с.

6. Филиповский В.М. Основы программирования и алгоритмизации. Практикум по алгоритмизации: Учебное пособие. – Санкт Петербург: СПбПУ, 2022. – 71 с.